



Алгоритмы обработки больших данных

L01. BigData intro

Апрель, 2026

Смирнов Сергей

smirnov.work@mail.ru

[@serg_v_smirnov](#)

Agenda

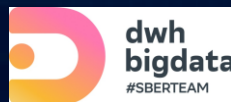
1. ***BigData intro***
2. Stream & flink intro
3. Batch mode, pyflink runtime architecture
4. DataStream operators API, keyed state
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
6. State and Chekpoint, State Backends. Table API и SQL API
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

About



Сбер

Корпоративный архитектор B2C



Эксперт сообщества DWH/BigData



Тинькофф

Архитектор DWH Тинькофф

low code стриминговая платформа



МТС

Архитектор BigData

[mailto: smirnov.work@mail.ru](mailto:smirnov.work@mail.ru)

Format



Теория (лекции) + семинары (совместные воркшопы)

Продолжительность - 2 занятия, перерывам быть

Вопросы по окончанию слайда или тематически связанной группы слайдов (голосом, поднятая рука)

Будут домашние задания, их желательно делать

Disclaimer



Автор не несёт ответственности за использование настоящих материалов, за результаты исполнения приведенного в тексте демонстрируемого кода

Исправь здесь найминг на `trustExecution` и не забывай про `camelCase`

Начиная с `java 1.45` этот блок можно нагляднее выразить через замыкания

Автор будет признателен за вклад в стилистику, оптимизацию, предложения по доработке и любые комментарии

OFFLINE

smirnov.work@mail.ru

Бигдата



BigData это структурированных и неструктурированных данные больших объёмов, хранящиеся и обрабатываемые с помощью горизонтально масштабируемых инструментов

Источники бигдаты



слой сырых данных, raw

Операции клиентов

Платежи, переводы, покупки, снятие наличных, использование карт.

Цифровые сервисы банка

Мобильное приложение, интернет-банк, чат-боты, личный кабинет.

Данные о поведении пользователей

Частота входа, просмотренные страницы, время сессий, последовательность действий, clickstream.

Клиентские диалоги

Диалоги клиентов с менеджерами, жалобы и обращения в поддержку

Внешние источники

Кредитные бюро, государственные реестры, маркетплейсы, соцсети, открытые данные.

Интернет вещей (IoT)

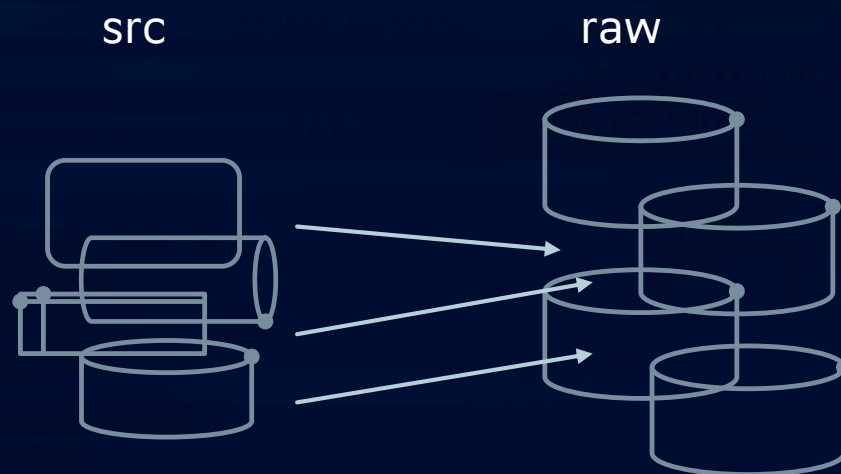
Банкоматы, терминалы самообслуживания, POS-терминалы, датчики в отделениях.

Финансовые транзакции и логи

Лог-файлы серверов, данные о мошеннических операциях, антифрод-системы

Источники бигдаты, слой raw

слой сырых данных, raw



Для чего используется бигдата



\$ чтобы больше заработать

Персонализация и кросс-продажи

Маркетинг (предложение клиентам релевантных продуктов)

Целевые программы (анализ потребностей на основе поведения)

Лояльность (повышение лояльности и удержание клиентов)

Улучшение клиентского опыта

Развитие мобильного приложения и интернет банкинга

Качество обслуживания

Скорость обработки операций, жалоб и предложений

Операционная эффективность

Автоматизация операций и процессов

Оптимизация работы отделений и банкоматов

\$ чтобы меньше тратить

Управление рисками и кредитный скоринг

Формирование индивидуальных кредитных предложений

Прогнозирование вероятности невозврата кредита

Противодействие мошенничеству

Выявление подозрительных операций

Поведенческие модели для выявления аномалий и снижение убытков от мошенничества

Соответствие регуляторным требованиям

Контроль соблюдения законодательства, отчетность и проверки

Слой витрин



DM (data mart)

Маркетинг и продажи

Кампании, триггеры, отклики, конверсии

Риски и кредитование

Скоринг, просрочки, управление лимитами

Клиенты

Сегментация, отток, CLTV

Транзакционная аналитика

RFM, категоризация расходов, тепловые карты, гео

Операционная деятельность

Производительность отделений, очереди, загрузка банкоматов

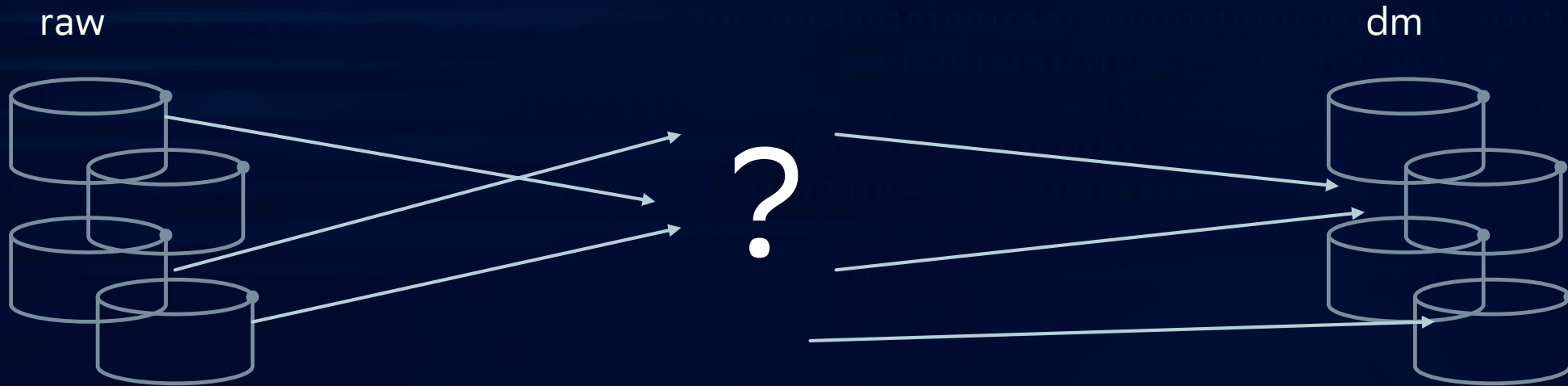
Регуляторная отчетность

МСФО, налоговая отчетность

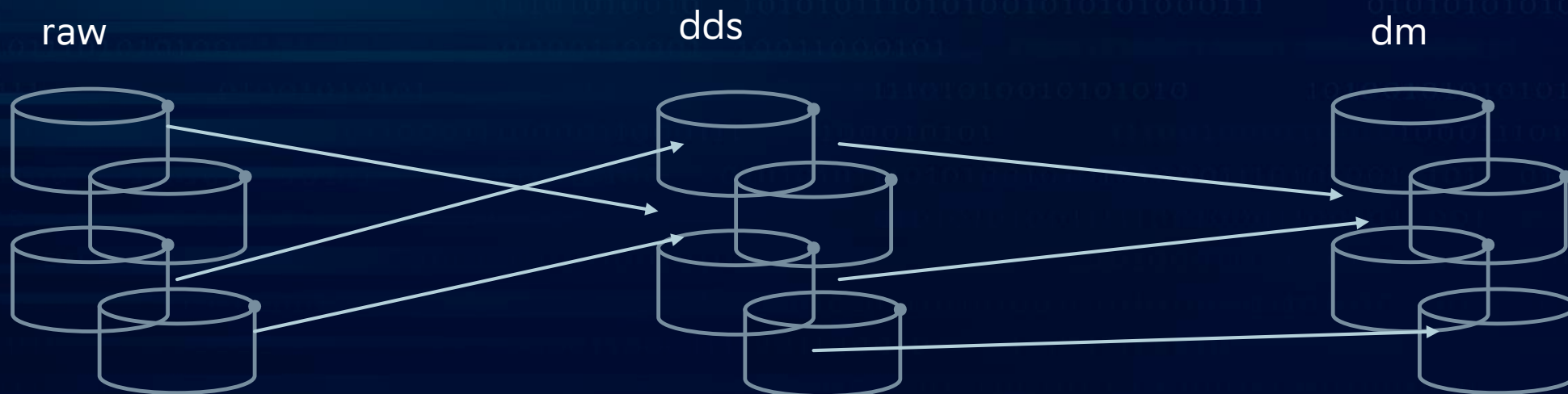
Комплаенс-контроль

Списки, согласия, правила

Data mart layer



Детальный слой



Преобразование, очистка и контроль качества

Нарезка на предметные области

Консолидация, объединение из нескольких источников

Фиксирование контракта (модель данных)

dds: широкая витрина клиента



Идентификационные
данные

Продуктовый
профиль

Транзакционное
поведение

Каналы
взаимодействия

Кредитная история
и риски

Поведенческие
паттерны

Внешние
данные



Широкий профиль клиента



Идентификационные данные

ФИО, контакты, демография, социальный статус, состав семьи.

Продуктовый профиль

Используемые продукты и услуги (карты, счета, кредиты, вклады, страхование), частота пользования, активность.

Транзакционное поведение

История платежей и переводов, средний чек, география трат, предпочитаемые категории (АЗС, супермаркеты, путешествия).

Каналы взаимодействия

Активность в мобильном приложении и интернет-банке, посещения отделений, обращения в кол-центр, реакция на коммуникации.

Кредитная история и риски

Скоринговый балл, наличие просрочек, кредитные нагрузки, участие в программах реструктуризации.

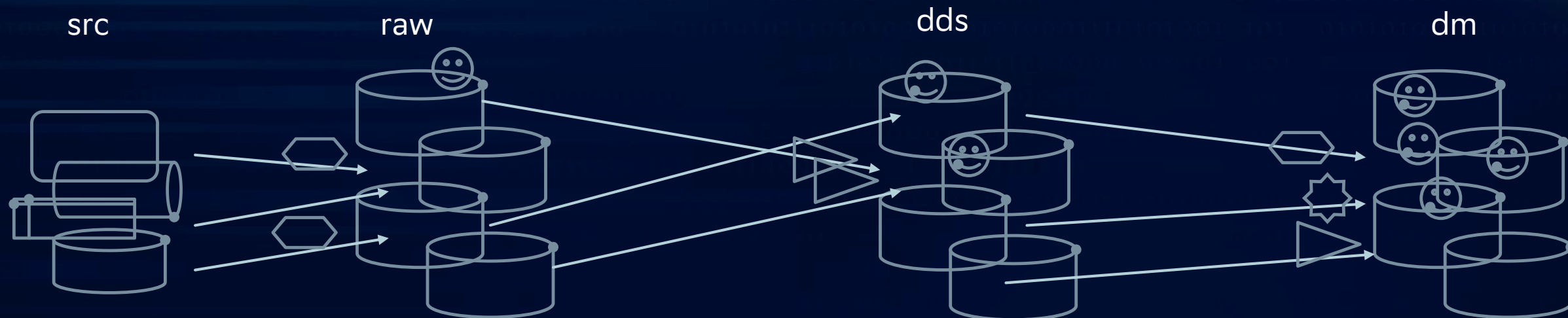
Поведенческие паттерны

Регулярность доходов, сезонность расходов, жизненные события (ипотека, автокредит), лояльность к бренду.

Внешние данные

Информация из государственных сервисов, соцсетей, партнёрских программ (с учётом требований законодательства).

Типовое хранилище



(!) Разделение по командам, инструментам, правилам и практикам

Источники

Реляционные

АБС

CRM

ДБО

Процессинг

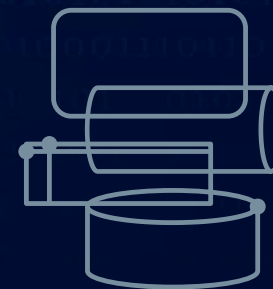
Документы

Brokerage

...

Строго структурированные
данные в виде таблиц
реляционных СУБД (postgres,
oracle, mysql, mssql, db2)

src



Не реляционные

Апи внутренних систем и открытые контракты внешних

Документы: аудио записи, сканы документов

Графы, временные ряды

Логи журналы и веб-серверов, приложений, баз данных, сетевого
оборудования

NoSQL базы данных

Объектные хранилища S3, hdfs,

...

Файлы структурированные
и неструктурированные:
json, xml, csv, zip, pdf

Не реляционные источники



Пример

zip архив, зашифрованный ГОСТом и подписанный квалифицированной подписью, отсканированного документа, нарезанного на части в несколько pdf документов внутри dbf-файла внутри blob-поля

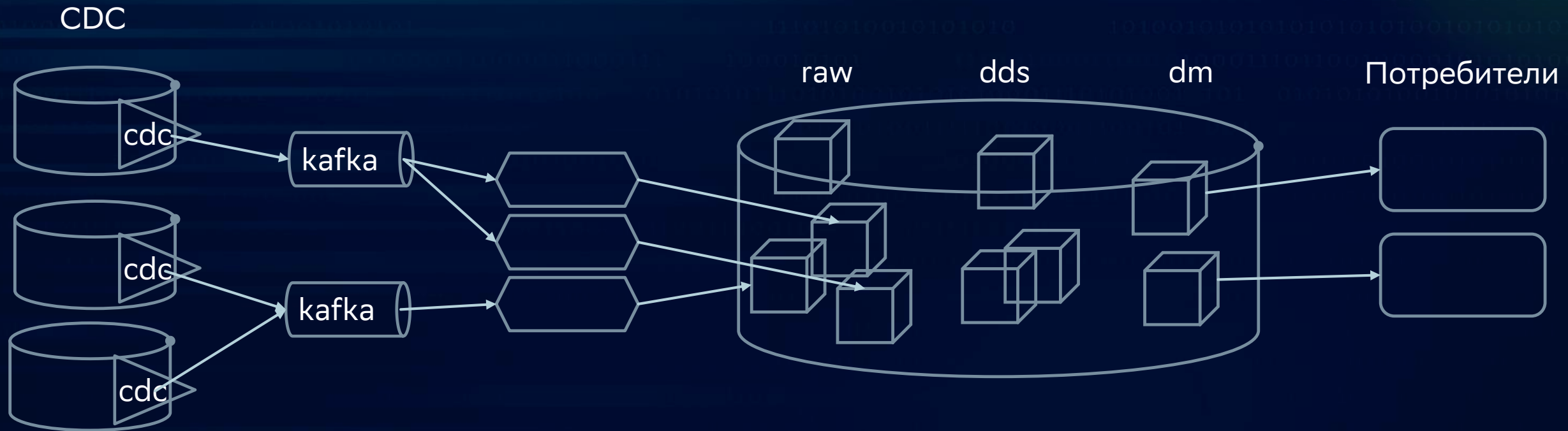
Инструмент

NiFi, IPC, Pentaho, dbt, python, java

Реализация

полноценный релиз -
аналитика, разработка,
тестирование и внедрение

Реляционные источники



CDC

create clients(id: PK, fio, age);

id	fio	age
----	-----	-----

insert into clients(1, "Иванов", 32);

id	fio	age
1	Иванов	32

insert into clients(2, "Петров", 27);

id	fio	age
1	Иванов	32
2	Петров	27

update clients set age=31 where id=1;

id	fio	age
1	Иванов	31
2	Петров	27

delete from clients where id=2

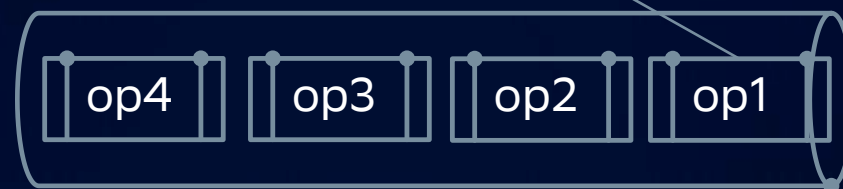
id	fio	age
1	Иванов	31

```
{  
  "before": null,  
  "after": {"id": 1, "fio": "Иванов", "age": 32},  
  "op": "I"  
}
```

```
{  
  "before": null,  
  "after": {"id": 2, "fio": "Петров", "age": 27},  
  "op": "I"  
}
```

```
{  
  "before": {"id": 1},  
  "after": {"id": 1, "age": 31},  
  "op": "U"  
}
```

```
{  
  "before": {"id": 2},  
  "after": null,  
  "op": "D"  
}
```



CDC (Change Data Capture) — технология захвата изменённых данных. Отслеживает и передает последние изменения (вставки, обновления, удаления) как правило на основе сведений из журнала транзакций

кафка, посредник



Задача

Петя возвращает наушники Саше

Решение 1, push

Петя в свое удобное время приносит наушники Саше

Архитектура

Петя ==push==> Саша

Проблематика

Саши неудобно или его нет дома

Решение 2, pull

Саша сам заходит за наушниками к Пете

Архитектура

Петя <==pull== Саша

Проблематика

Пете неудобно или его нет дома

Решение 3, broker

Петя относит наушники консьержке, Саша их забирает

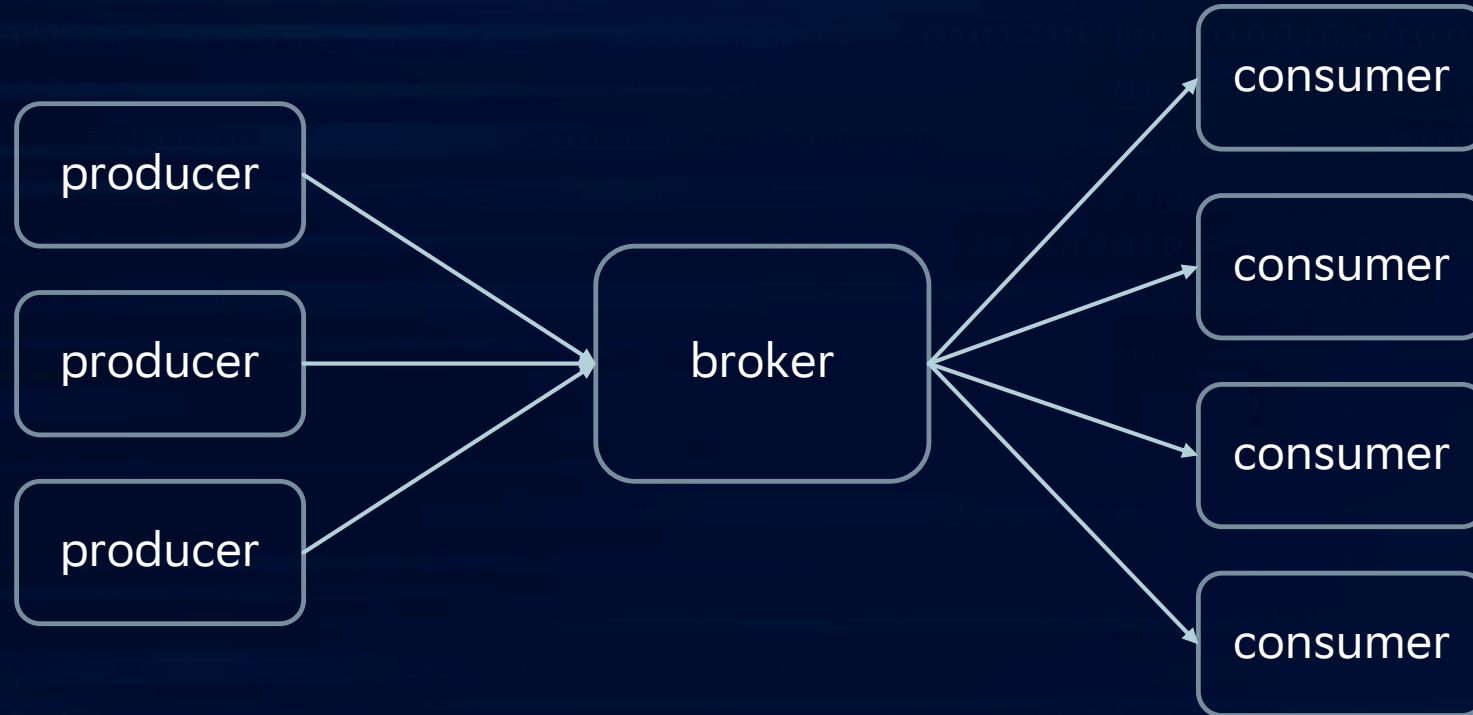
Архитектура

Петя =>broker=> Саша

Проблематика

Нужен посредник

kafka, broker



Полезные свойства

Быстро
Надежно
Масштабируемо
Прогнозируемо
Известно
Бесплатно
Доступно
Сопровождается
Процессно
Расширяемо

kafka, терминология



broker – компонент интеграционного взаимодействия, ответственный за получение, хранение и доставку сообщения

message или сообщение, набор байтов характеризующий объект или событие, сохраненное в заданном формате

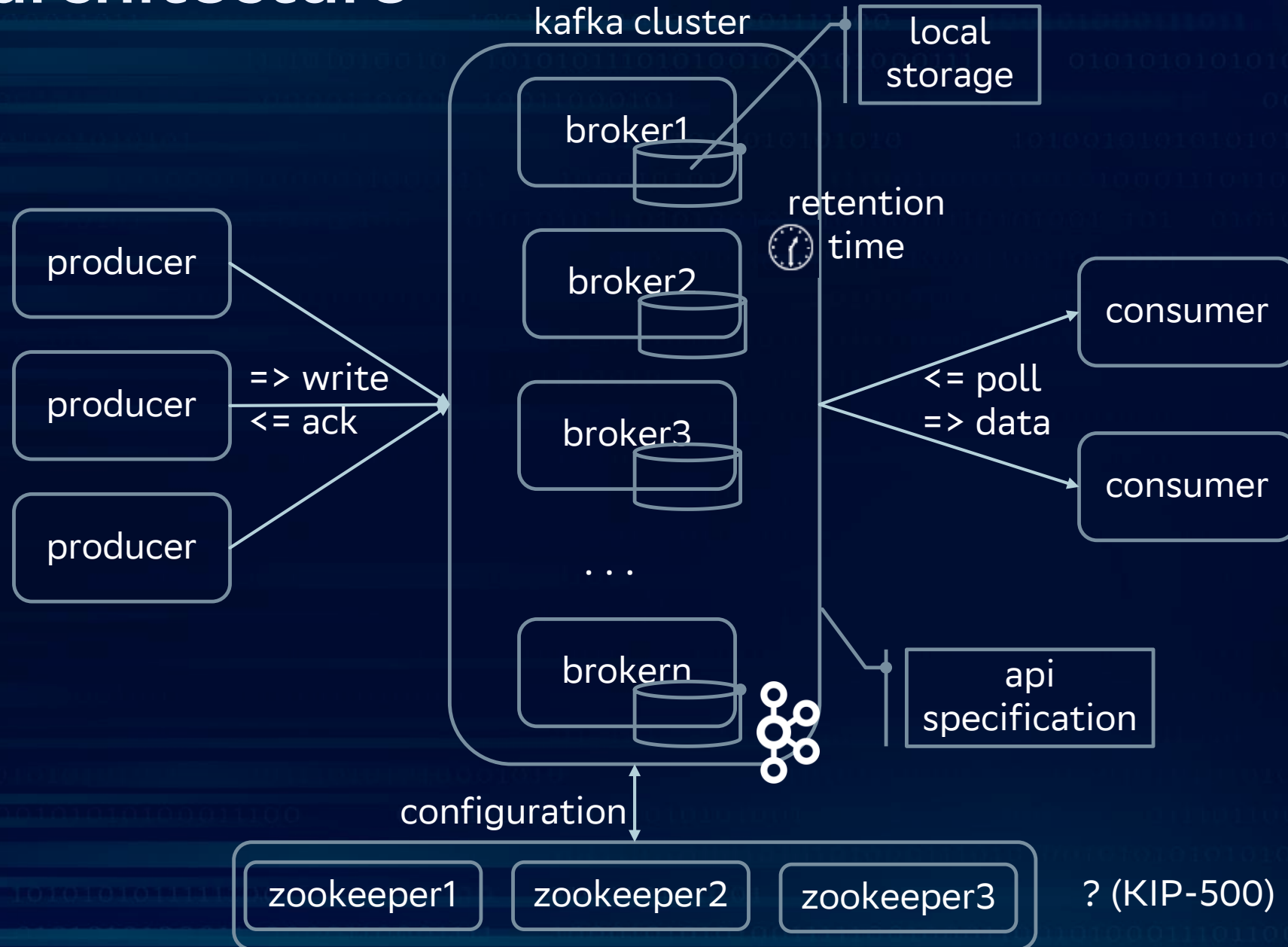
topic – способ разделения сообщений на логические группы. Каждому сообщению присваивается (правильнее сообщение публикуется) свой топик. Выбор топика осуществляет publisher. Другое определение – последовательность логически связанных сообщений

publisher – система-отправитель сообщений

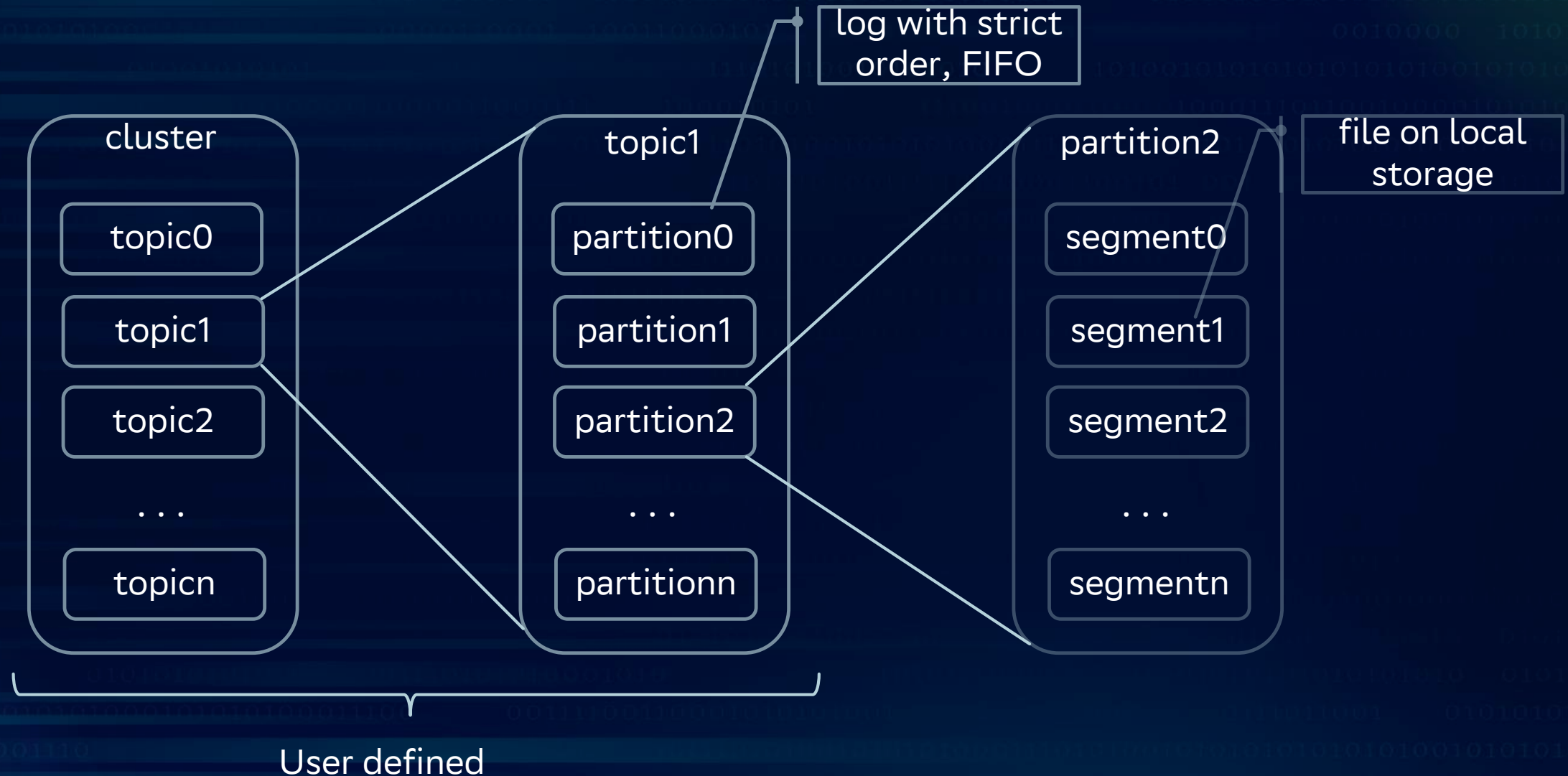
subscriber – система-получатель сообщений

topic partition – единица масштабирования топика

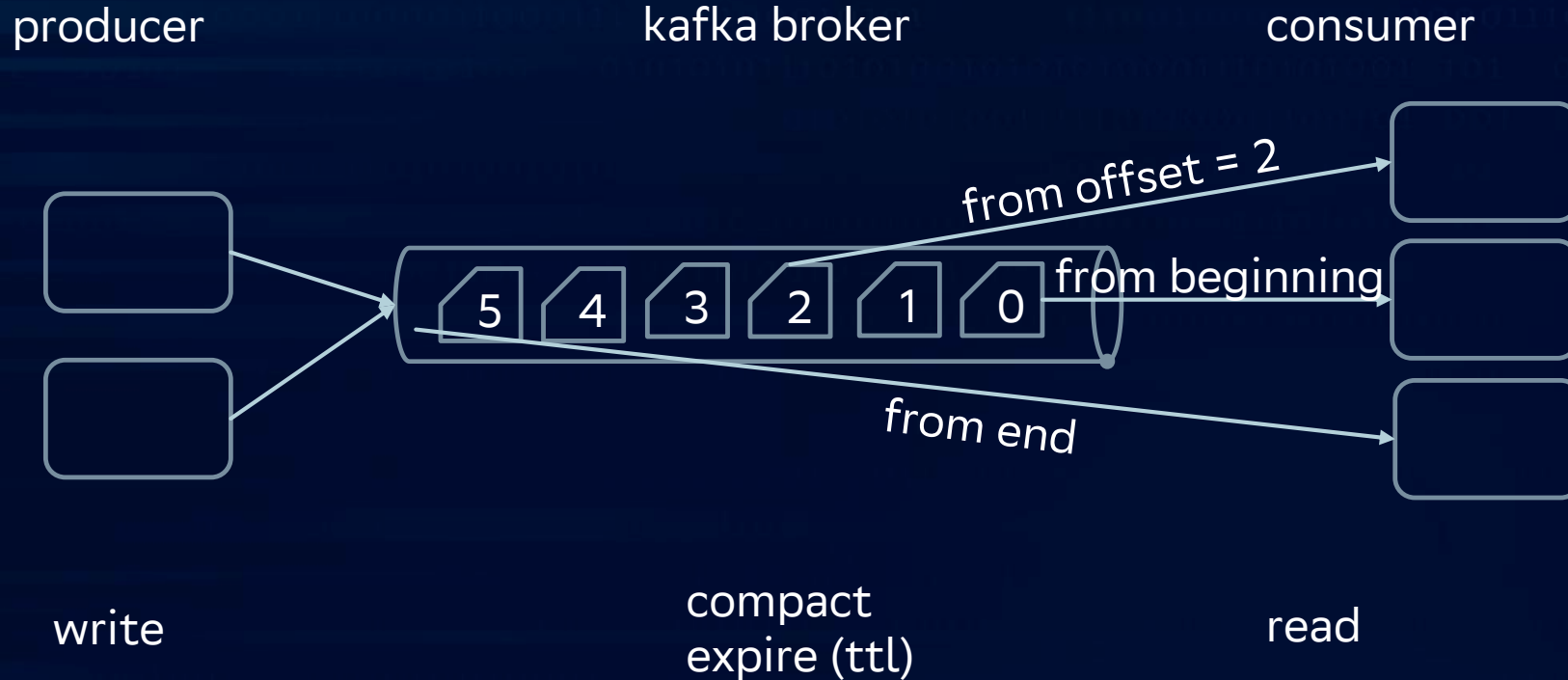
kafka, architecture



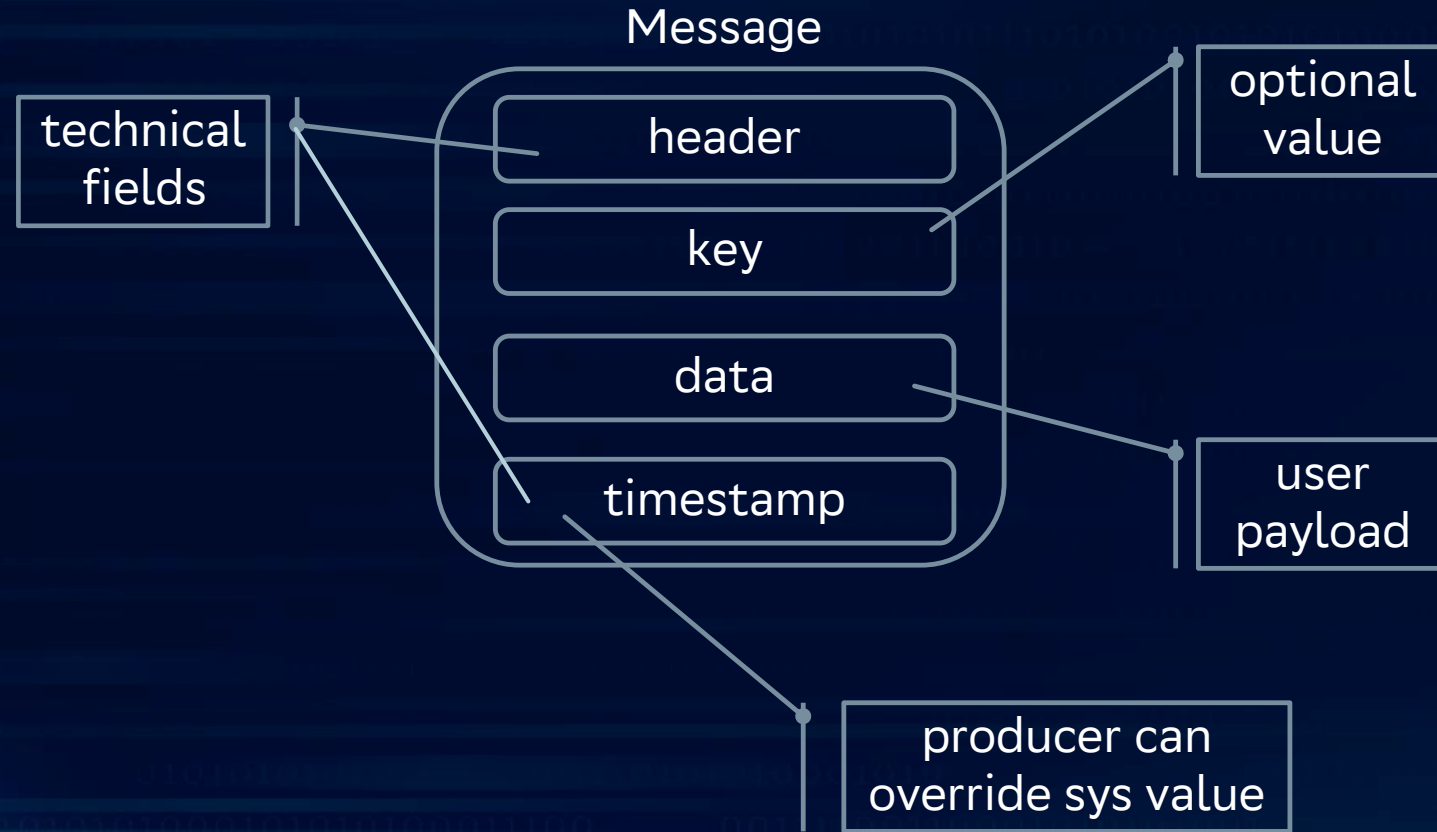
kafka, topic, partiotion, segment



kafka, log operations



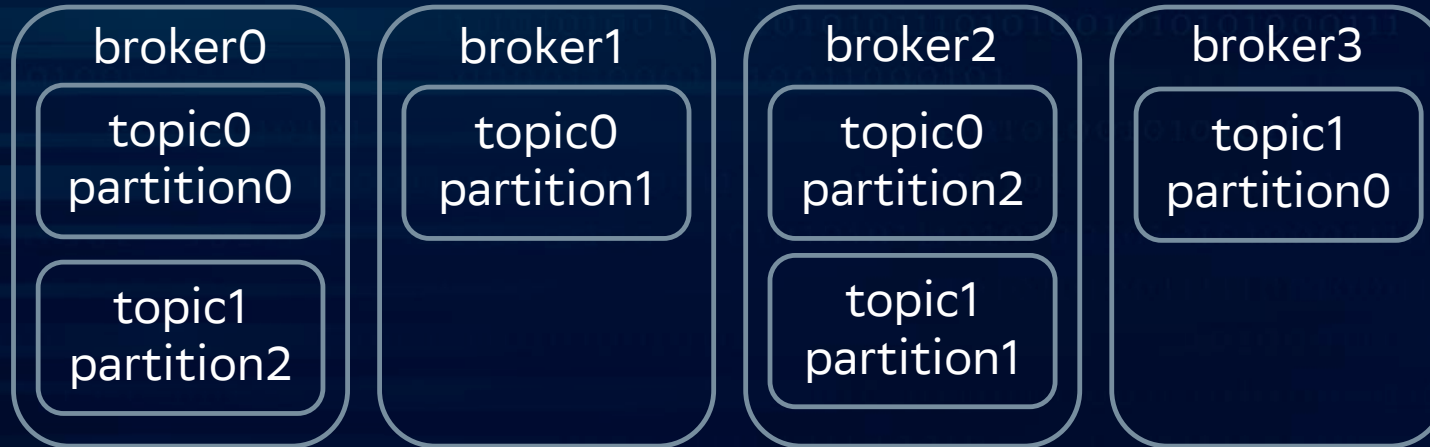
kafka, data format



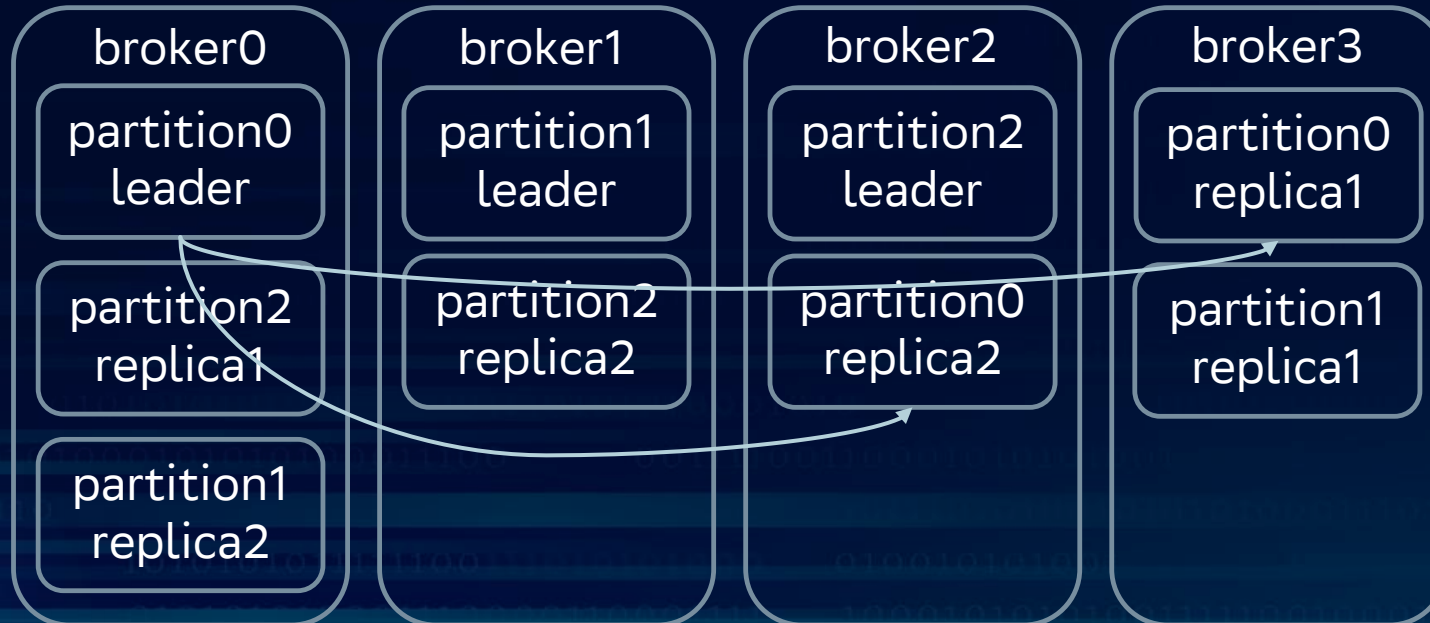
kafka, deployment



4 brokers; 2 topics; 3 partition per topic



4 brokers; 1 topic; 3 partition; rf=3

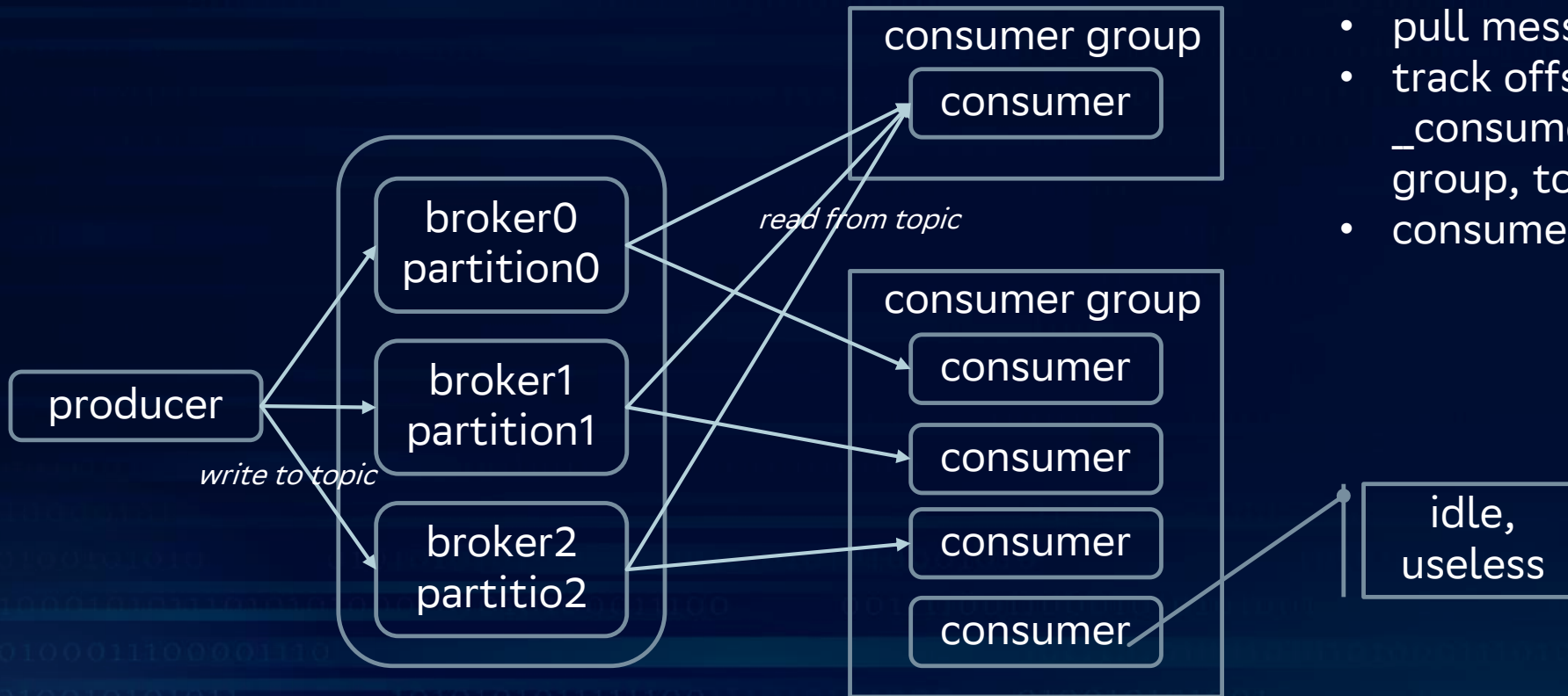


kafka, partitions



Producer partition strategy

- Key exists -> default, $\text{hash}(\text{key}) \% \text{partition_number}$
- No key -> round robin
- Custom partitioner



Consumer

- pull messages from all topics
- track offset in special topic `_consumer_offsets` per consumer group, topic, partition
- consumer partitioner strategy

kafka, configurations



Topic

partition count
replication factor
ttl
topic compaction
min.insync.replicas

producer
acks = all

Producer

broker
acks = 0, 1, all
enable.idempotence
transactional.id

exactly
once conf

Consumer

broker
groupId
auto.commit
isolation.level

Example, python + cdc

```
import psycopg2

conn = psycopg2.connect("dbname=testdb user=postgres password=postgres host=localhost")
cur = conn.cursor()

sql = "select * from pg_logical_slot_get_changes('my_slot', NULL, NULL);"

cur.execute(sql)
rows = cur.fetchall()

for row in rows:
    print(row)

conn.commit()
conn.close()
```

Example, python + kafka

```
from kafka import KafkaConsumer
import json

TOPIC_NAME = "test_flink"
KAFKA_BROKER = '127.0.0.1:9092'

consumer = KafkaConsumer(
    *topics: TOPIC_NAME,
    bootstrap_servers=[KAFKA_BROKER],
    auto_offset_reset='earliest', # Start reading from the beginning of the topic earliest, latest
    enable_auto_commit=True,
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)

print(f"Listening for messages on topic: {TOPIC_NAME}")

#consume and print messages
for message in consumer:
    print(f"Received message: {message.value}, topic: {message.topic}, partition: {message.partition}")

#close the consumer connection (this part might not be reached in a continuous loop in Colab)
#consumer.close()
```

Homework



kafka + python

- *Развернуть кластер kafka*
- *Написать простое приложение по чтению и записи в kafka*

Questions



Спасибо за внимание

d-people vs dev-people



dev work

```
private static final Logger LOGGER = Logger.getLogger( ClassName.class.getName() );
LOGGER.log( Level.FINE, starting work for {0} entries in loop", list.size() );
try {
    ds.flatMap(new FlatMapFunction<String, Tuple2<String, Integer>>() {
        @Override
        public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {
            for(String word: value.split(" ")){
                out.collect(new Tuple2(word, 1));
            }
        }
    }).keyBy("word").print();
} catch( Exception ex ) {
    LOGGER.log( Level.SEVERE, ex.toString(), ex );
}
@RequestMapping("/srvExpose")
public void endpoint() {
    String processingResult = trnCount(trnExpose);
    trnExpose.labels("trn count",processTrn).inc();
}
```

de work

```
private static final Logger LOGGER = Logger.getLogger( ClassName.class.getName() );
LOGGER.log(1, "started at : " + LocalDate.now());
ds.flatMap(new FlatMapFunction<String, Tuple2<String, Integer>>() {
    @Override
    public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {
        for(String word: value.split(" ")){
            out.collect(new Tuple2(word, 1));
        }
    }
}).keyBy("word").print();
LOGGER.log(1, "end at : " + LocalDate.now());
```

ds work

```
st1.flatMap(new FlatMapFunction<String, Tuple2<String, Integer>>() {  
    @Override  
    public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {  
        for(String w2: value.split(" ")){  
            out.collect(new Tuple2(w2, 1));  
        }  
    }  
}).keyBy(0).print();
```